

Java Full Stack Viva Guide

Spring & Spring Boot Track — Questions with Model Answers

Core Java

Spring Core

Spring Boot

Spring Data JPA

REST APIs

Spring Security

Microservices

Full Stack Integration

Build Tools & Git

Project Questions

Day-Wise Preparation Schedule (Aug 19 → Sep 18, 2026)

Built from your QuantumCrew Java Full Stack + Gen AI day-wise training plan. Sundays are excluded. Cloud (AWS) and DevOps topics (Terraform, Docker, Kubernetes, Jenkins, SonarQube deep-dive) are intentionally given minimal time since they carry less weight for a Java Full Stack viva — only a short awareness-level pass is scheduled for them. Each row tells you which section of this PDF to study that day.

Day	Date	Focus Topic	PDF Section
1	Wed, Aug 19	Core Java basics: OOPs, JDK/JRE/JVM, overloading vs overriding	Sec 1 (1.1–1.7)
2	Thu, Aug 20	Exception handling, multithreading, String handling, generics	Sec 1 (1.7, 1.12–1.15) + Sec 4
3	Fri, Aug 21	Collections Framework + Java 8 (lambdas, streams, functional interfaces)	Sec 1 (1.8–1.11) + Sec 4
4	Sat, Aug 22	Java 11/17 highlights, SOLID principles, Design patterns	Sec 5
—	Sun, Aug 23	No study — rest day	
5	Mon, Aug 24	JDBC & MySQL basics + Hibernate/JPA (ORM, entity lifecycle)	Sec 6
6	Tue, Aug 25	Spring Core: IoC, DI, bean scopes/lifecycle, AOP	Sec 2
7	Wed, Aug 26	Spring Data JPA: repositories, query methods, @Query	Sec 7
8	Thu, Aug 27	Spring MVC: controllers, ModelAndView, validation	Sec 8
9	Fri, Aug 28	REST principles, RestTemplate, DTOs, HTTP status codes	Sec 9
10	Sat, Aug 29	Revision 1: Core Java + Spring Core + JPA quick recall	Sec 1, 2, 6, 7
—	Sun, Aug 30	No study — rest day	
11	Mon, Aug 31	Spring Boot fundamentals: auto-config, starters, @SpringBootApplication	Sec 3 (3.1–3.7)
12	Tue, Sep 1	Spring Boot REST APIs, global exception handling, profiles	Sec 3 (3.8–3.14)
13	Wed, Sep 2	Microservices concepts, Actuator, Feign, Swagger (Spring Cloud kept brief)	Sec 10
14	Thu, Sep 3	Spring Security: authentication, JWT, filter chain	Sec 11
15	Fri, Sep 4	Testing: JUnit 5, Mockito, TDD basics	Sec 12
16	Sat, Sep 5	Revision 2: Spring Boot + Security + Testing + Microservices	Sec 3, 10, 11, 12
—	Sun, Sep 6	No study — rest day	
17	Mon, Sep 7	Frontend foundations: HTML, CSS, JavaScript basics	Sec 13
18	Tue, Sep 8	React fundamentals: components, props, state, hooks	Sec 14
19	Wed, Sep 9	React + Spring Boot integration: Axios, useState/useEffect	Sec 14 (14.7–14.10)
20	Thu, Sep 10	Maven build lifecycle + Git fundamentals	Sec 15
21	Fri, Sep 11	Agile/Scrum basics + brief Cloud/DevOps awareness (low priority)	Sec 16
22	Sat, Sep 12	Revision 3: Frontend + React + Maven/Git	Sec 13, 14, 15
—	Sun, Sep 13	No study — rest day	
23	Mon, Sep 14	Project defense prep: architecture walkthrough, your role	Sec 17
24	Tue, Sep 15	Project defense prep: challenges, DB design, testing, deployment	Sec 17
25	Wed, Sep 16	Full mock viva: mixed questions across all sections, note weak spots	All
26	Thu, Sep 17	Final revision: re-read only the ★ high-priority questions	All ★ items
27	Fri, Sep 18 — VIVA DAY	Light glance at ★ questions only. No new topics. Sleep well the night before.	—

How to Use This Guide

★ **HIGH PRIORITY** — These questions are asked in almost every Spring Boot viva, across colleges and companies. Know these cold.

CORE CONCEPT (blue background) — Foundational questions. If you get one wrong, the panel usually digs deeper here, so these deserve extra practice.

Plain (white background) — Good to know, commonly asked as follow-ups or in rounds that go slightly deeper.

The questions are grouped the same way most trainers structure a Java Full Stack (Spring Boot) viva: Core Java fundamentals first, then Spring Core concepts, then Spring Boot specifics, then data access, REST, security, microservices, frontend integration, tooling, and finally project-defense questions. Most panels spend 60–70% of the time on Spring Boot + REST + JPA + your project, so prioritize those sections if you're short on time.

1. Core Java Fundamentals

These set the foundation. Even in a Spring Boot-focused viva, panels often start here to check basics before moving to frameworks.

1.1 What is the difference between JDK, JRE, and JVM?

JVM (Java Virtual Machine) is the runtime engine that executes bytecode — it makes Java platform-independent. **JRE (Java Runtime Environment)** = JVM + libraries needed to *run* Java applications (no compiler). **JDK (Java Development Kit)** = JRE + development tools like the compiler (`javac`), debugger, etc. — needed to *write and compile* Java code. So the relationship is: $\text{JDK} \supset \text{JRE} \supset \text{JVM}$.

Asked in almost every interview as an opener — answer confidently and quickly.

1.2 Explain the four pillars of OOP with examples.

- **Encapsulation** — wrapping data (fields) and methods together, hiding internal state using private fields and public getters/setters.
- **Inheritance** — a class (subclass) acquires properties/behaviors of another class (superclass) using `extends`.
- **Polymorphism** — one interface, many implementations. Compile-time (method overloading) and runtime (method overriding).
- **Abstraction** — hiding implementation details and exposing only functionality, via abstract classes or interfaces.

Be ready to give a real code example for each, not just definitions.

1.3 Difference between an abstract class and an interface. When would you use which?

An abstract class can have both abstract and concrete methods, constructors, and instance variables, but supports single inheritance only. An interface (pre-Java 8) could only have abstract methods; from Java 8 onward it can also have `default` and `static` methods, and a class can implement multiple interfaces. Use an abstract class when classes share a common base implementation; use an interface when you just want to define a contract/capability (e.g., `Comparable`, `Runnable`) that unrelated classes can implement.

1.4 Difference between method overloading and overriding.

Overloading is compile-time polymorphism — same method name, different parameter list, within the same class. **Overriding** is runtime polymorphism — a subclass provides a specific implementation of a method already defined in its superclass, with the same signature.

1.5 Difference between `==` and `.equals()`.

`==` compares references (memory addresses) for objects, and actual values for primitives. `.equals()` compares the actual content/value of objects, and can be overridden to define custom equality (e.g., `String` overrides it to compare characters, not references).

Common follow-up: "What happens if you override `equals()` but not `hashCode()`?"

1.6 What is the `hashCode()` and `equals()` contract?

If two objects are equal according to `equals()`, they must return the same `hashCode()`. The reverse isn't required (two objects with the same hash code need not be equal — that's a "collision"). Breaking this contract causes bugs when objects are used as keys in `HashMap` or elements in `HashSet`, because lookups rely on both methods working together.

1.7 Checked vs unchecked exceptions.

Checked exceptions are checked at compile time (e.g., `IOException`, `SQLException`) — the method must either handle them with try-catch or declare them with `throws`. **Unchecked exceptions** (subclasses of `RuntimeException`, e.g., `NullPointerException`, `ArrayIndexOutOfBoundsException`) are not checked at compile time — they usually indicate programming bugs.

1.8 What is a functional interface? Give examples.

An interface with exactly one abstract method (it can have multiple default/static methods). It's the basis for lambda expressions. Examples: `Runnable` (`run()`), `Comparator<T>` (`compare()`), and the ones in `java.util.function`: `Function<T,R>`, `Predicate<T>`, `Supplier<T>`, `Consumer<T>`.

Directly relevant because Spring Boot code uses lambdas heavily (streams, bean definitions).

1.9 Explain lambda expressions and the Streams API with an example.

A lambda expression is a concise way to represent an anonymous function, e.g. `(a, b) -> a + b`. The Streams API (Java 8+) lets you process collections declaratively using operations like `filter`, `map`, `collect`, `reduce`, instead of writing explicit loops. Example:

```
list.stream().filter(x -> x > 10).map(x -> x * 2).collect(Collectors.toList())
```

1.10 Difference between `ArrayList` and `LinkedList` .

`ArrayList` is backed by a dynamic array — fast random access ($O(1)$ get), but slow insertion/deletion in the middle ($O(n)$ due to shifting). `LinkedList` is a doubly linked list — fast insertion/deletion ($O(1)$ once you have the node), but slow random access ($O(n)$).

1.11 `HashMap` vs `TreeMap` vs `LinkedHashMap` .

`HashMap` — no ordering guarantee, $O(1)$ average access, allows one null key. `LinkedHashMap` — maintains insertion order, slightly more overhead than `HashMap`. `TreeMap` — sorted by key (natural ordering or a custom `Comparator`), $O(\log n)$ operations, implemented as a Red-Black tree.

1.12 What is multithreading? Difference between `Runnable` and `Thread` .

Multithreading allows concurrent execution of two or more parts of a program for maximum CPU utilization. You can create a thread by extending `Thread` (but Java doesn't support multiple inheritance, so this "uses up" your one superclass) or by implementing `Runnable` and passing it to a `Thread` object (preferred, since it separates the task from the thread mechanism and allows implementing other interfaces too).

1.13 What is garbage collection in Java?

Automatic memory management where the JVM identifies and removes objects that are no longer reachable from any live thread, freeing heap memory. Java developers don't manually free memory (unlike C/C++). Common GC algorithms include Serial, Parallel, CMS, and G1 (default in modern JVMs).

1.14 Difference between `String` , `StringBuilder` , and `StringBuffer` .

`String` is immutable — every modification creates a new object. `StringBuilder` is mutable and not thread-safe (faster). `StringBuffer` is mutable and thread-safe (synchronized methods, so slightly slower). Use `StringBuilder` in single-threaded code (most common case); use `StringBuffer` only when multiple threads modify the same string.

1.15 What are generics and why are they used?

Generics let you write classes, interfaces, and methods that operate on a type specified as a parameter, e.g. `List<String>` . They provide compile-time type safety (catching type errors early) and eliminate the need for explicit casting.

2. Spring Core (IoC, DI, Beans, AOP)

This is the conceptual backbone of Spring Boot. Panels use these to check whether you understand what's happening "under the hood" of annotations you use daily.

2.1 What is the Spring Framework and why use it?

Spring is a lightweight, open-source framework for building Java applications, built around **Inversion of Control** and **Dependency Injection**. It reduces boilerplate, promotes loose coupling between components, and offers modules for data access (JDBC/JPA), web (MVC), security, transactions, and more — letting developers focus on business logic instead of plumbing code.

2.2 What is Inversion of Control (IoC) and Dependency Injection (DI)?

IoC is a design principle where the control of object creation and lifecycle is transferred from the application code to a container/framework, instead of objects creating their own dependencies. **DI** is how Spring implements IoC — the Spring container "injects" required dependencies into a class (via constructor, setter, or field) rather than the class instantiating them itself. This decouples classes and makes them easier to test (mock dependencies) and maintain.

The single most-asked Spring concept — be able to explain it with a plain (non-Spring) example too, e.g., a Car needing an Engine.

2.3 Types of dependency injection in Spring.

- **Constructor injection** — dependencies passed via constructor; recommended because it allows immutability and makes dependencies mandatory/explicit.
- **Setter injection** — dependencies set via setter methods; useful for optional dependencies.
- **Field injection** — `@Autowired` directly on a field; concise but harder to test/unit-mock and hides dependencies, so it's discouraged in production code.

2.4 What is the Spring IoC container? Difference between `BeanFactory` and `ApplicationContext`.

The IoC container is responsible for instantiating, configuring, and managing the lifecycle of beans. `BeanFactory` is the basic container (lazy initialization). `ApplicationContext` is a more advanced container built on top of `BeanFactory` — it supports eager initialization, internationalization, event propagation, AOP integration, and is what almost all real applications (including Spring Boot) use.

2.5 Explain the Spring Bean lifecycle.

Roughly: container reads configuration → instantiates the bean → injects dependencies (DI) → calls `@PostConstruct` / `InitializingBean.afterPropertiesSet()` → bean is ready to use → on shutdown, `@PreDestroy` / `DisposableBean.destroy()` is called before the bean is removed.

2.6 What are Spring bean scopes?

- **singleton** (default) — one instance per Spring container.
- **prototype** — a new instance every time the bean is requested.
- **request** — one instance per HTTP request (web-aware context).
- **session** — one instance per HTTP session.

2.7 Difference between `@Component`, `@Service`, `@Repository`, and `@Controller`.

All are stereotype annotations that mark a class as a Spring-managed bean (all are meta-annotated with `@Component`), but they add semantic meaning and extra behavior:

- `@Component` — generic stereotype for any Spring-managed bean.
- `@Service` — marks the service/business logic layer (semantic only, no extra behavior).
- `@Repository` — marks the DAO/persistence layer; also enables automatic translation of persistence-specific exceptions into Spring's `DataAccessException` hierarchy.
- `@Controller` — marks a Spring MVC controller that returns views; `@RestController` = `@Controller` + `@ResponseBody`, returning data (JSON) directly.

Extremely commonly asked — know that they're functionally similar but semantically different, and that `@Repository` does exception translation.

2.8 How does `@Autowired` resolve ambiguity when multiple beans of the same type exist?

By default, Spring throws a `NoUniqueBeanDefinitionException` if it finds more than one matching bean. You resolve it using `@Qualifier("beanName")` to specify exactly which bean to inject, or `@Primary` on one of the bean definitions to mark it as the default choice.

2.9 What is Spring AOP? Explain aspect, advice, pointcut, and join point.

AOP (Aspect-Oriented Programming) lets you modularize cross-cutting concerns (logging, security, transactions) separately from business logic.

- **Aspect** — a module encapsulating a cross-cutting concern (a class annotated `@Aspect`).
- **Advice** — the action taken (e.g., `@Before` , `@After` , `@Around`).
- **Pointcut** — an expression that matches which join points the advice applies to.
- **Join point** — a point during execution (typically a method call) where an aspect can be applied.

Real use case: `@Transactional` itself is implemented using Spring AOP proxies.

2.10 Difference between XML-based and annotation-based configuration.

XML configuration (`beans.xml`) defines beans and wiring externally — verbose but keeps configuration separate from code.

Annotation-based configuration (`@Configuration` , `@Bean` , `@ComponentScan` , stereotype annotations) is more concise and colocated with the code. Spring Boot almost entirely uses annotation/Java-based configuration with minimal XML.

3. Spring Boot Specifics

This is usually the biggest chunk of questions since it's the "headline" tech of your stream.

3.1 What is Spring Boot and how is it different from Spring?

Spring Boot is built on top of the Spring Framework to simplify application setup and development. Plain Spring requires a lot of manual configuration (XML/Java config, dependency versions, server setup). Spring Boot provides **auto-configuration**, **starter dependencies** (pre-configured dependency bundles), an **embedded server** (Tomcat/Jetty, so no need to deploy a WAR separately), and production-ready features via **Actuator** — letting you get a working application running with minimal setup ("convention over configuration").

Almost guaranteed to be asked — have a crisp, confident 20-second answer ready.

3.2 What is auto-configuration in Spring Boot?

Spring Boot automatically configures beans based on the dependencies (JARs) present on the classpath and properties defined in `application.properties` / `application.yml`. For example, if `spring-boot-starter-data-jpa` and an H2 driver are on the classpath, Spring Boot automatically configures a `DataSource`, `EntityManagerFactory`, and transaction manager — without you writing that configuration manually. It's driven by `@EnableAutoConfiguration` and conditional annotations like `@ConditionalOnClass`.

3.3 Explain the `@SpringBootApplication` annotation. What does it bundle internally?

It's a convenience annotation combining three annotations:

- `@Configuration` — marks the class as a source of bean definitions.
- `@EnableAutoConfiguration` — enables Spring Boot's auto-configuration mechanism.
- `@ComponentScan` — scans the current package and sub-packages for Spring-managed components.

Very frequently asked as a direct follow-up to "what is Spring Boot".

3.4 What is a starter dependency? Give an example.

A starter is a curated set of dependencies bundled together for a specific purpose, so you don't have to manually manage individual library versions. Example: `spring-boot-starter-web` pulls in Spring MVC, an embedded Tomcat server, and Jackson (for JSON) all at once. Other common ones: `spring-boot-starter-data-jpa`, `spring-boot-starter-security`, `spring-boot-starter-test`.

3.5 What is `application.properties` / `application.yml` used for?

It's the central place to externalize configuration — server port, database URL/credentials, logging levels, custom application properties, active profile, etc. — without hardcoding values in Java code. YAML (`.yml`) is more readable for nested/hierarchical configuration; properties files use flat dot-notation keys.

3.6 How do you change the default embedded server or port?

Set `server.port=8081` in `application.properties` (default is 8080). To switch the embedded server (e.g., from Tomcat to Jetty/Undertow), exclude the default starter's server dependency (`spring-boot-starter-tomcat`) and add the alternative starter (e.g., `spring-boot-starter-jetty`) in `pom.xml`.

3.7 What is Spring Boot Actuator?

A production-readiness module that exposes operational endpoints for monitoring and managing the app — e.g. `/actuator/health`, `/actuator/metrics`, `/actuator/info`, `/actuator/env`. Useful for health checks, metrics collection (often integrated with Prometheus/Grafana), and debugging in production.

3.8 Difference between `@Controller` and `@RestController`.

`@Controller` is used for traditional MVC apps that return view names (resolved to HTML/JSP pages by a view resolver).

`@RestController` = `@Controller` + `@ResponseBody` on every method — the return value is written directly to the HTTP response body (typically serialized to JSON), which is what REST APIs need.

3.9 Explain `@RequestMapping`, `@GetMapping`, `@PostMapping`, etc.

`@RequestMapping` maps HTTP requests to handler methods/classes and can specify the HTTP method via an attribute (`method = RequestMethod.GET`). `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, `@PatchMapping` are shorthand, method-specific versions introduced for readability — e.g. `@GetMapping("/users")` is equivalent to `@RequestMapping(value="/users", method=RequestMethod.GET)`.

3.10 How do you handle exceptions globally in Spring Boot?

Using `@ControllerAdvice` (or `@RestControllerAdvice` for REST APIs) on a class combined with `@ExceptionHandler` methods for specific exception types. This centralizes error handling instead of repeating try-catch in every controller, and lets you return consistent, structured error responses (e.g., a custom `ErrorResponse` object with status, message, timestamp).

Panels love this one because it shows real project maturity — mention it even if not asked directly.

3.11 `@RequestBody` VS `@ResponseBody` VS `@PathVariable` VS `@RequestParam`.

- `@RequestBody` — binds the HTTP request body (typically JSON) to a Java object.
- `@ResponseBody` — serializes the return value directly into the HTTP response body.
- `@PathVariable` — extracts a value from the URI path, e.g. `/users/{id}`.
- `@RequestParam` — extracts a value from query parameters, e.g. `/users?name=John`.

3.12 How do you enable CORS in Spring Boot?

At the method/class level with `@CrossOrigin(origins = "http://localhost:3000")`, or globally by implementing `WebMvcConfigurer` and overriding `addCorsMappings()` to define allowed origins, methods, and headers application-wide.

3.13 JAR vs WAR — how do you package and run a Spring Boot app?

Spring Boot apps are typically packaged as an executable JAR (with an embedded server bundled in) using `mvn package`, then run with `java -jar app.jar` — no external server needed. A WAR is used only if you need to deploy to an external servlet container (like a standalone Tomcat), which requires extending `SpringBootServletInitializer`.

3.14 What are Spring profiles?

Profiles let you maintain separate configurations for different environments (dev, test, prod) — e.g. `application-dev.yml`, `application-prod.yml`. You activate one via `spring.profiles.active=dev`, and beans can be restricted to a profile using `@Profile("dev")`.

4. Collections, Java 8/11/17 & Multithreading Deep Dive

Covers Day 2–3 of your schedule: the Collections API, modern Java features, and concurrency basics.

4.1 What is the Collections Framework? Name its core interfaces.

A unified architecture for storing and manipulating groups of objects. Core interfaces: `List` (ordered, allows duplicates — `ArrayList`, `LinkedList`), `Set` (no duplicates — `HashSet`, `TreeSet`), `Map` (key-value pairs — `HashMap`, `TreeMap`), and `Queue / Deque` (FIFO/double-ended — `ArrayDeque`, `PriorityQueue`).

4.2 How does a `HashMap` work internally?

It stores entries in an array of "buckets." The key's `hashCode()` determines the bucket index. On collision (two keys map to the same bucket), entries are stored as a linked list (or a balanced tree if a bucket gets too large, since Java 8) within that bucket. Lookup computes the hash, jumps to the bucket, then uses `equals()` to find the exact match.

4.3 What are default and static methods in interfaces (Java 8)? Why were they introduced?

Default methods let an interface provide a method body that implementing classes can use as-is or override. **Static methods** belong to the interface itself and are called on the interface name, not an instance. They were introduced so existing interfaces (like those in the Collections framework) could add new methods without breaking every class that already implements them.

4.4 What are method references? Give an example.

A shorthand for a lambda that only calls an existing method, using the `::` syntax. Example: `list.forEach(System.out::println)` is equivalent to `list.forEach(x -> System.out.println(x))`. Types include static method references (`ClassName::staticMethod`), instance method references (`object::instanceMethod`), and constructor references (`ClassName::new`).

4.5 Explain `map()` vs `flatMap()` in Streams.

`map()` transforms each element into another single element (1-to-1), e.g. converting a stream of Strings to their lengths. `flatMap()` transforms each element into a stream and flattens all those streams into one (1-to-many), useful when each element is itself a collection — e.g. flattening a `List<List<String>>` into a single `Stream<String>`.

4.6 What's new/notable in Java 11 and Java 17 (LTS versions)?

Java 11: new `String` methods (`isBlank()`, `strip()`, `lines()`), the standardized `HttpClient` API, the `var` keyword usable in lambda parameters, and a single-file source-code launcher (`java HelloWorld.java` without compiling first). **Java 17:** sealed classes (restrict which classes can extend/implement a type), enhanced pattern matching for `switch` (preview), and stronger encapsulation of JDK internals. Both are Long-Term Support (LTS) releases, which is why enterprises standardize on them.

4.7 What is a race condition? How do you prevent it in Java?

A race condition occurs when two or more threads access shared data concurrently and the final outcome depends on the unpredictable timing/order of execution, leading to incorrect results. Prevented using synchronization — the `synchronized` keyword (method or block), `Lock` objects from `java.util.concurrent.locks`, or thread-safe classes like `ConcurrentHashMap` and `AtomicInteger`.

4.8 What is the difference between `synchronized` method and `synchronized` block?

A `synchronized method` locks the entire method on the object's (or class's, for static methods) monitor for its whole duration. A `synchronized block` locks only the specific critical section of code on a chosen object, giving finer-grained control and better performance since less code is locked.

4.9 What is the `try-with-resources` statement?

A try statement that declares one or more resources (implementing `AutoCloseable`) which are automatically closed at the end of the block, in reverse order of creation — removing the need for an explicit `finally` block to close streams, connections, etc.

4.10 What is the difference between `throw` and `throws`?

`throw` is used inside a method body to actually raise/instantiate an exception, e.g. `throw new IllegalArgumentException("bad input")`. `throws` is used in a method signature to declare that the method might propagate a checked exception to its caller, e.g. `public void readFile() throws IOException`.

5. SOLID Principles & Design Patterns

Panels ask these to check if you understand "why" your code is structured a certain way, not just "how" to write it.

5.1 Explain the SOLID principles.

- **S** — Single Responsibility: a class should have only one reason to change.
- **O** — Open/Closed: classes should be open for extension but closed for modification.
- **L** — Liskov Substitution: subtypes must be substitutable for their base types without breaking behavior.
- **I** — Interface Segregation: prefer many small, specific interfaces over one large general-purpose one.
- **D** — Dependency Inversion: depend on abstractions, not concrete implementations (this is exactly what Spring's DI enables).

Tie the "D" back to Spring's Dependency Injection when answering — it shows you connect theory to practice.

5.2 What is the Singleton design pattern? How does Spring use it?

Singleton ensures a class has only one instance and provides a global point of access to it. Spring beans are singleton-scoped by default — the Spring IoC container itself manages creating and returning that single shared instance, so you don't need to hand-write the classic private-constructor Singleton pattern yourself.

5.3 What is the Factory design pattern?

A creational pattern where object creation logic is centralized in a "factory" method/class instead of using `new` directly in client code, decoupling object creation from usage. Spring's `BeanFactory` / `ApplicationContext` is itself a large-scale implementation of this pattern for creating beans.

5.4 What is the Builder pattern? Where have you seen it in Spring/Java?

Builder separates the construction of a complex object from its representation, letting you build it step by step with a fluent API. Examples: `UriComponentsBuilder` in Spring, `StringBuilder`, and Lombok's `@Builder` annotation used commonly on entity/DTO classes.

5.5 What is the difference between creational, structural, and behavioral design patterns?

Creational patterns deal with object creation (Singleton, Factory, Builder). **Structural** patterns deal with composing classes/objects into larger structures (Adapter, Decorator, Proxy). **Behavioral** patterns deal with communication/responsibility between objects (Observer, Strategy, Template Method).

5.6 What is the Proxy pattern, and how does Spring AOP relate to it?

Proxy provides a placeholder/surrogate object that controls access to another object. Spring AOP implements cross-cutting concerns (like `@Transactional`, logging) by creating runtime proxies (JDK dynamic proxies for interfaces, or CGLIB proxies for classes) around your beans, which intercept method calls to add the extra behavior before/after the real method executes.

6. JDBC, Hibernate & JPA

Foundation for the "how does data get from Java to the database" line of questioning.

6.1 What is JDBC?

Java Database Connectivity — a standard Java API for connecting to and executing queries against relational databases. Typical flow: load driver → open `Connection` → create `Statement` / `PreparedStatement` → execute query → process `ResultSet` → close resources.

6.2 Difference between `Statement` and `PreparedStatement`.

`Statement` executes static SQL and is compiled every time it's run — vulnerable to SQL injection if you concatenate user input.

`PreparedStatement` is precompiled, accepts parameters via placeholders (`?`), is faster for repeated execution, and is safe from SQL injection since parameters are bound, not concatenated.

Always mention SQL injection prevention — interviewers specifically listen for this.

6.3 What is ORM? What problem does Hibernate/JPA solve?

Object-Relational Mapping maps Java objects to database tables automatically, so developers work with objects instead of writing raw SQL for every operation. It solves the "impedance mismatch" between the object-oriented world (classes, inheritance, references) and the relational world (tables, rows, foreign keys) — handling CRUD, caching, lazy loading, and transaction management for you.

6.4 What is JPA vs Hibernate?

JPA (Java Persistence API) is just a specification/set of interfaces (`@Entity` , `EntityManager` , etc.) — it defines *what* should happen, not how. **Hibernate** is the most popular concrete implementation of that specification — it provides the actual code that makes JPA work. Spring Data JPA sits on top of both, further reducing boilerplate.

6.5 Explain the entity lifecycle in JPA/Hibernate.

Four states: **Transient** (a plain new object, not associated with any persistence context or DB row), **Persistent/Managed** (attached to a session, changes are auto-tracked and synced to DB), **Detached** (was persistent but the session closed — changes are no longer tracked), and **Removed** (marked for deletion from the DB).

6.6 What is lazy loading vs eager loading?

Lazy loading (`FetchType.LAZY`) delays fetching related/associated data until it's actually accessed — better performance, but can cause a `LazyInitializationException` if accessed after the session closes. **Eager loading** (`FetchType.EAGER`) fetches associated data immediately along with the parent entity, which is simpler but can hurt performance by pulling in unnecessary data.

`@ManyToOne` / `@OneToOne` default to EAGER; `@OneToMany` / `@ManyToMany` default to LAZY.

6.7 Explain `@OneToMany` , `@ManyToOne` , `@ManyToMany` , `@OneToOne` .

These map relational foreign-key relationships to object references. E.g. one `Department` has many `Employee` s (`@OneToMany` on `Department` , `@ManyToOne` on `Employee` , which is the "owning side" holding the foreign key). `@ManyToMany` needs a join table (e.g. `Students & Courses`). `@OneToOne` is a 1:1 mapping (e.g. `User & UserProfile`).

6.8 What is JPQL? How is it different from SQL?

JPQL (Java Persistence Query Language) is an object-oriented query language — you query against entity names and their fields (e.g. `SELECT e FROM Employee e WHERE e.salary > 50000`) rather than table/column names. Hibernate translates JPQL into actual SQL for the underlying database, making queries database-agnostic.

6.9 What strategies exist for auto-generating a primary key?

Set via `@GeneratedValue(strategy = ...)` : `IDENTITY` (DB auto-increment column), `SEQUENCE` (uses a DB sequence object, good for batching), `TABLE` (uses a separate table to simulate a sequence, portable but slower), and `AUTO` (lets the provider pick based on the DB dialect).

6.10 What is the first-level cache vs second-level cache in Hibernate?

First-level cache is enabled by default, scoped to a single `Session` / `EntityManager` — it avoids duplicate DB hits for the same entity within one session. **Second-level cache** is optional, shared across sessions/the whole `SessionFactory` (often backed by Ehcache/Redis), reducing DB load for frequently read, rarely changed data.

7. Spring Data JPA

The layer that connects Spring apps to the database with minimal boilerplate — a favorite viva area since it's used in almost every project.

7.1 Difference between `CrudRepository`, `JpaRepository`, and `PagingAndSortingRepository`.

`CrudRepository` provides basic CRUD methods (`save`, `findById`, `deleteById`, etc.). `PagingAndSortingRepository` extends it, adding pagination and sorting support. `JpaRepository` extends both and adds JPA-specific features like batch operations (`saveAll`) and `flush()`. In practice, most Spring Boot projects just extend `JpaRepository` since it includes everything.

7.2 How do you write custom queries in Spring Data JPA?

Two main ways: **Query Method naming convention** — Spring parses the method name to generate the query automatically, e.g. `findByLastNameAndAge(String lastName, int age)`. Or the **@Query annotation** — write JPQL or native SQL explicitly, e.g. `@Query("SELECT e FROM Employee e WHERE e.salary > :sal")`, using named (`:sal`) or positional (`?1`) parameters.

7.3 What is the N+1 query problem and how do you avoid it?

It happens when fetching a list of N parent entities triggers one query for the parents plus N additional queries (one per parent) to lazily fetch each one's associated child data — hugely inefficient. Avoided using `JOIN FETCH` in JPQL, `@EntityGraph`, or by explicitly fetching related data in a single query instead of relying on lazy loading in a loop.

7.4 What is `@Transactional` and why is it important?

It marks a method (or class) so that all database operations inside it run within a single transaction — either all changes commit together, or all roll back together if an exception occurs, preserving data consistency (ACID). Without it, a partial failure could leave the database in an inconsistent state (e.g. money debited from one account but not credited to another).

7.5 Difference between `save()`, `saveAndFlush()`, and `deleteById()`.

`save()` persists/updates an entity, but the actual SQL may be deferred until the transaction commits or the persistence context is flushed. `saveAndFlush()` immediately synchronizes the change to the database, useful when you need the DB state updated right away (e.g. before a native query). `deleteById()` removes the entity matching the given ID.

8. Spring MVC

Even though most modern apps are REST-first, panels still test the underlying MVC concepts.

8.1 Explain the Spring MVC request flow (DispatcherServlet architecture).

A request first hits the **DispatcherServlet** (front controller), which consults a **HandlerMapping** to find the right controller method, invokes it (possibly passing through interceptors), gets back a `ModelAndView`, then asks a **ViewResolver** to resolve the logical view name to an actual view (e.g. a JSP), which is rendered and returned as the response.

8.2 What is `ModelAndView` ?

An object that carries both the model data (attributes to display) and the view name (which page to render) back from a controller method to the DispatcherServlet, which then hands off to the ViewResolver.

8.3 What's the difference between parent and child (WebApplicationContext) contexts in Spring MVC?

The **root/parent context** holds shared beans like services, repositories, and DAOs, accessible application-wide. The **child/web context** (WebApplicationContext) holds web-specific beans like controllers and view resolvers, and can access beans from the parent context but not vice versa — keeping web-layer concerns isolated from business-layer beans.

8.4 How do you perform validation in Spring MVC/Boot?

Using Bean Validation (JSR-380) annotations on model fields — `@NotNull`, `@NotBlank`, `@Size`, `@Email`, `@Min` / `@Max` — combined with `@Valid` on the controller method parameter to trigger validation. Errors are captured in a `BindingResult` object, or (in REST APIs) automatically converted to a 400 Bad Request handled via `@ExceptionHandler(MethodArgumentNotValidException.class)`. Custom validation is done by creating a custom annotation with a `ConstraintValidator` implementation.

9. REST APIs & Web Services

This overlaps with Spring Boot but focuses on REST as a concept — interviewers often test this independently of framework specifics.

9.1 What is REST? Explain its core principles.

REST (Representational State Transfer) is an architectural style for designing networked applications using standard HTTP methods. Core principles: **statelessness** (server holds no client session state between requests), **client-server separation**, **uniform interface** (consistent resource-based URIs), **cacheability**, and a **layered system**.

9.2 Difference between REST and SOAP.

REST is an architectural style using lightweight formats (usually JSON) over HTTP, stateless, and flexible. SOAP is a strict protocol using XML only, has built-in standards for security/transactions (WS-Security), and is heavier/more verbose. REST is generally preferred today for web/mobile APIs; SOAP is still used in some enterprise/legacy or highly regulated (banking) systems.

9.3 Explain HTTP methods and their idempotency.

GET — retrieve data, safe & idempotent. **POST** — create a resource, NOT idempotent (calling it twice creates two resources). **PUT** — replace/update a resource fully, idempotent (same result if repeated). **PATCH** — partially update, not guaranteed idempotent. **DELETE** — remove a resource, idempotent (deleting twice has the same end state).

"Idempotent" is a favorite trap question — know exactly which methods are and aren't.

9.4 Common HTTP status codes and what they mean.

200 OK — success. **201 Created** — resource created (typical POST response). **400 Bad Request** — invalid client input. **401 Unauthorized** — authentication missing/failed. **403 Forbidden** — authenticated but not permitted. **404 Not Found** — resource doesn't exist. **500 Internal Server Error** — unexpected server-side failure.

9.5 How do you version a REST API?

Common approaches: **URI versioning** (`/api/v1/users`) — simplest and most common; **request parameter** (`/users?version=1`); **custom header** (`X-API-Version: 1`); or **content negotiation** via the `Accept` header. URI versioning is the most widely used because it's explicit and easy to test.

9.6 What is a DTO and why use it instead of exposing entities directly?

A Data Transfer Object is a plain object used to carry data between layers/across the network, decoupled from the JPA entity. It prevents exposing internal DB structure or sensitive fields to API consumers, avoids lazy-loading serialization issues, and lets you shape the API response independently of how data is stored.

9.7 How do you document/test REST APIs?

Document with **Swagger/OpenAPI** (springdoc-openapi generates interactive docs automatically from annotated controllers). Test manually with **Postman**, and automate with **MockMvc** or `@SpringBootTest` combined with `TestRestTemplate` / `WebTestClient` for integration tests.

10. Microservices & Spring Cloud (Concepts)

Kept concept-focused rather than deployment-heavy, since infra/cloud tooling gets less priority for this viva.

10.1 Monolithic vs Microservice architecture.

A **monolith** is a single deployable unit containing all functionality — simple to develop/deploy initially, but hard to scale specific parts and risky to change (one bug can bring down everything). **Microservices** split the application into small, independently deployable services, each owning its own data and business capability — easier to scale and deploy independently, but adds complexity (network calls, distributed data, monitoring).

10.2 What is service discovery? What role does Eureka play?

In a microservices setup, service instances can scale up/down and change addresses dynamically, so services can't hardcode each other's URLs. A **service registry** (like Netflix Eureka) lets services register themselves on startup and lets other services look them up by name at runtime, enabling dynamic discovery instead of static configuration.

10.3 What is an API Gateway?

A single entry point that sits in front of all microservices, routing requests to the appropriate service, and often handling cross-cutting concerns like authentication, rate limiting, and load balancing centrally instead of duplicating that logic in every service.

10.4 What is a Circuit Breaker (Resilience4j)? Why is it needed?

A pattern that prevents an application from repeatedly trying to call a failing/slow downstream service. After a threshold of failures, the circuit "opens" and further calls fail fast (or return a fallback) instead of waiting/retrying, giving the failing service time to recover and preventing cascading failures across the system.

10.5 What is OpenFeign?

A declarative REST client — instead of manually writing HTTP call boilerplate (like with `RestTemplate`), you define an interface with annotations describing the endpoint, and Feign generates the implementation at runtime. It integrates well with service discovery (call services by name) and load balancing.

10.6 How do microservices communicate with each other?

Synchronous — REST calls (`RestTemplate`/`Feign`/`WebClient`) or gRPC, where the caller waits for a response. **Asynchronous** — message queues/brokers (Kafka, RabbitMQ), where services publish events and other services consume them independently, improving decoupling and resilience.

10.7 What is Spring Boot Actuator used for in a microservices context?

It exposes health, metrics, and info endpoints (`/actuator/health` , `/actuator/metrics`) for each microservice, which monitoring tools and orchestrators use to check if a service instance is healthy and should keep receiving traffic.

11. Spring Security

Almost always asked once you mention REST APIs — "how did you secure your endpoints?" is a standard follow-up.

11.1 Difference between authentication and authorization.

Authentication verifies *who* the user is (e.g. checking username/password or a token's validity). **Authorization** determines *what* an authenticated user is allowed to do (e.g. checking if they have the "ADMIN" role to access a particular endpoint). Authentication always happens first.

11.2 What is JWT and how is it used for stateless authentication?

JSON Web Token is a compact, self-contained, digitally signed token containing claims (user info, roles, expiry) encoded in three parts: header, payload, signature. After login, the server issues a JWT to the client; the client sends it in the `Authorization: Bearer <token>` header on every subsequent request. Since the token itself carries all needed info and is verified via its signature, the server doesn't need to store session state — making it ideal for stateless REST APIs and microservices.

Be ready to explain WHY it's stateless (no server-side session lookup needed) — that's the actual point interviewers are testing.

11.3 Explain the Spring Security filter chain.

Every incoming request passes through a chain of `Filter` objects before reaching the `DispatcherServlet` — each filter handles one concern (e.g. `UsernamePasswordAuthenticationFilter` for form login, a custom JWT filter to validate tokens, `ExceptionHandlerFilter` for handling security exceptions). If a filter can't authenticate the request and the resource is protected, the chain short-circuits and returns 401/403.

11.4 How do you implement role-based access control in Spring Security?

Using method-level security annotations like `@PreAuthorize("hasRole('ADMIN')")` on controller/service methods (after enabling with `@EnableMethodSecurity`), or by configuring URL-based rules in the `SecurityFilterChain` bean, e.g.

```
.requestMatchers("/admin/**").hasRole("ADMIN")
```

11.5 How are passwords stored securely in Spring Security?

Never in plain text. Spring Security provides `BCryptPasswordEncoder`, which hashes the password with a random salt built in, making it resistant to rainbow-table attacks and computationally expensive to brute-force. Passwords are hashed on registration and compared (hash-to-hash) on login — never decrypted back to plain text.

11.6 What is CSRF? Does it apply to REST APIs?

Cross-Site Request Forgery tricks an authenticated user's browser into submitting a malicious request unknowingly, exploiting cookie-based session authentication. It's mainly a concern for browser/cookie-session-based apps. For stateless REST APIs authenticated via JWT in headers (not cookies), CSRF protection is typically disabled since there's no session cookie to hijack.

12. Testing (JUnit 5 & Mockito)

Shows the panel you write reliable, professional code — always a good area to sound confident in.

12.1 What is unit testing and why is it important?

Testing individual units (usually a single method/class) in isolation from the rest of the system, typically by mocking dependencies. It catches bugs early, documents expected behavior, gives confidence when refactoring, and forms the base of the "testing pyramid" (unit > integration > end-to-end in volume).

12.2 Common JUnit 5 annotations.

`@Test` — marks a test method. `@BeforeEach` / `@AfterEach` — run before/after every test method. `@BeforeAll` / `@AfterAll` — run once before/after all tests in the class (must be static). `@Disabled` — skips a test. `@ParameterizedTest` — runs the same test logic with different input values.

12.3 What is Mockito? Why do we mock dependencies?

Mockito is a framework for creating mock (fake) objects that simulate the behavior of real dependencies, using `@Mock` and `when(...).thenReturn(...)`. We mock dependencies so a unit test only tests the class under test's own logic, without needing a real database, network call, or other slow/unreliable external system.

12.4 Difference between `@Mock` and `@InjectMocks`.

`@Mock` creates a mock instance of a dependency (e.g. a mocked `UserRepository`). `@InjectMocks` creates a real instance of the class under test (e.g. `UserService`) and automatically injects the `@Mock` objects into it (via constructor/setter/field), so you can test the service's real logic against fake dependencies.

12.5 What is Test-Driven Development (TDD)?

A development approach where you write a failing test first (Red), write the minimum code to make it pass (Green), then refactor the code while keeping the test passing (Refactor) — the "Red-Green-Refactor" cycle. It forces you to think about requirements/edge cases before implementation.

12.6 Difference between unit testing and integration testing in Spring Boot.

Unit tests test a class in isolation with mocked dependencies (fast, no Spring context loaded). **Integration tests** (using `@SpringBootTest`) load the actual Spring application context (or a slice of it, e.g. `@WebMvcTest`, `@DataJpaTest`) to verify multiple components work correctly together, often against a real or in-memory (H2) database.

13. Frontend Foundations (HTML, CSS, JavaScript)

Quick-recall questions to prove basic frontend competency before React.

13.1 Difference between `id` and `class` attributes in HTML/CSS.

`id` must be unique within a page and is used to target exactly one element (higher CSS specificity); `class` can be reused across multiple elements and is used for shared styling.

13.2 Difference between `var`, `let`, and `const` in JavaScript.

`var` is function-scoped and hoisted (can be redeclared, leads to bugs). `let` is block-scoped and can be reassigned. `const` is block-scoped and cannot be reassigned after declaration (though object/array contents it points to can still be mutated). Modern JS favors `const` by default and `let` when reassignment is needed.

13.3 What is the CSS Box Model?

Every HTML element is a rectangular box made of, from inside out: **content** → **padding** (space inside the border) → **border** → **margin** (space outside the border, between elements).

13.4 What is the DOM?

The Document Object Model is a tree-like, in-memory representation of an HTML page that JavaScript can read and manipulate — adding/removing elements, changing content or styles, and responding to events, which is how dynamic web pages work.

13.5 What is event bubbling in JavaScript?

When an event fires on an element, it first triggers on that element, then "bubbles up" and triggers the same event on each of its ancestor elements in turn, up to the document root — unless stopped with `event.stopPropagation()`.

13.6 What is JSON and why is it commonly used with REST APIs?

JavaScript Object Notation — a lightweight, human-readable, language-independent data format (key-value pairs, arrays). It's the default format REST APIs use because it maps naturally to JavaScript objects on the frontend and is easy for most languages (including Java, via Jackson) to serialize/deserialize.

14. React & Spring Boot Integration

Directly tied to your training's frontend-integration module — expect at least one question connecting frontend and backend.

14.1 What is React? Why use it?

A JavaScript library for building user interfaces using a component-based architecture. It uses a **virtual DOM** to efficiently update only the parts of the real DOM that changed (instead of re-rendering the whole page), leading to better performance, and encourages reusable, composable UI components.

14.2 Difference between functional and class components.

Class components use ES6 classes, extend `React.Component`, and manage state via `this.state / this.setState()`. **Functional components** are plain JavaScript functions that use **Hooks** (`useState`, `useEffect`) to manage state and side effects. Functional components with Hooks are the modern standard — simpler syntax, easier to test and reuse logic.

14.3 What is the difference between props and state?

Props (properties) are read-only data passed from a parent component to a child — the child cannot modify them. **State** is data managed internally within a component, which can change over time (via `useState`) and triggers a re-render when updated.

14.4 What is `useState()` ?

A Hook that lets a functional component hold and update local state. `const [count, setCount] = useState(0)` gives you the current value (`count`) and a function (`setCount`) to update it — calling the setter triggers React to re-render the component with the new value.

14.5 What is `useEffect()` ? When does it run?

A Hook for handling side effects (API calls, subscriptions, timers, manually touching the DOM) in functional components. It runs after render. Its behavior depends on the dependency array: no array = runs after every render; empty array `[]` = runs once, after the initial render (like `componentDidMount`); array with values = runs whenever any of those values change.

Classic follow-up: "what happens if you forget the dependency array?" — know it causes an infinite re-render loop if you're also calling `setState` inside it.

14.6 What is a key prop in React lists and why is it needed?

A unique identifier given to each element when rendering a list (e.g. `.map()`), so React can efficiently track which items changed, were added, or removed between renders, instead of re-rendering the entire list from scratch.

14.7 How do you consume a Spring Boot REST API from React?

Typically using **Axios** (or the Fetch API) inside a `useEffect()` hook to call the endpoint when the component mounts, then storing the response in state via `useState()` to trigger a re-render with the fetched data. Example pattern: `useEffect(() => { axios.get('/api/users').then(res => setUsers(res.data)); }, [])`.

14.8 How do you handle CORS errors when React (localhost:3000) calls Spring Boot (localhost:8080)?

Since they're different origins (different ports), the browser blocks the request by default. Fix it on the backend by adding `@CrossOrigin(origins = "http://localhost:3000")` on the controller, or by configuring a global CORS policy via `WebMvcConfigurer`, explicitly allowing the frontend's origin.

14.9 How do you handle form submission and client-side validation in React?

Controlled components bind each input's value to state (via `onChange` handlers updating `useState`), so React always holds the current form data. On submit, you validate the state values (checking required fields, formats, etc.) before calling the API, showing inline error messages for any that fail, and only calling the backend once client-side checks pass (though server-side validation is still required for real security).

14.10 What is a Single Page Application (SPA)?

A web app that loads a single HTML page and dynamically updates content via JavaScript as the user interacts, instead of requesting a full new page from the server on every navigation — giving a faster, more app-like experience. React (with React Router) is commonly used to build SPAs that consume a REST API backend.

15. Maven & Git

Tooling questions — usually quick, factual, and asked to confirm you actually used these day-to-day.

15.1 What is Maven? What problem does it solve?

A build automation and dependency management tool for Java projects. It solves the problem of manually downloading and managing JAR files and their versions — you declare dependencies in `pom.xml` and Maven downloads them (and their transitive dependencies) automatically from a repository, plus standardizes the build lifecycle (compile, test, package, install, deploy).

15.2 What is `pom.xml` ? Name its key sections.

Project Object Model — Maven's core configuration file. Key sections: `<groupId>` / `<artifactId>` / `<version>` (project coordinates), `<dependencies>` (required libraries), `<build>` / `<plugins>` (build behavior), and `<parent>` (for Spring Boot, usually `spring-boot-starter-parent`, which manages compatible dependency versions for you).

15.3 Difference between `mvn install` and `mvn package` .

`mvn package` compiles the code, runs tests, and bundles it into a JAR/WAR in the local `target/` folder. `mvn install` does everything `package` does, plus additionally copies that built artifact into your local `.m2` repository, making it available as a dependency for other local Maven projects.

15.4 Basic Git commands you'd use daily.

`git clone` (copy a remote repo locally), `git add` (stage changes), `git commit` (save staged changes locally), `git push` (send commits to remote), `git pull` (fetch + merge remote changes), `git branch` (create/list branches), `git checkout -b` (create and switch to a new branch), `git merge` (combine branches).

15.5 What is a merge conflict? How do you resolve it?

It occurs when Git can't automatically reconcile changes made to the same lines of a file on two different branches being merged. Git marks the conflicting sections in the file with `<<<<<<`, `=====`, `>>>>>>` markers; you manually edit the file to keep the correct/combined content, remove the markers, then `git add` the resolved file and complete the merge with `git commit` .

15.6 What is a pull request?

A request to merge changes from one branch (often a feature branch, possibly on a fork) into another (often `main`), used on platforms like GitHub/GitLab to enable code review and discussion before the changes are actually merged into the target branch.

16. Agile Basics & Cloud/DevOps Awareness (Light)

Kept brief and concept-only, as requested — these carry lower priority for this viva than the Java/Spring/React sections above. Enough to answer confidently if asked, not deep implementation detail.

16.1 What is Agile methodology? What is Scrum?

Agile is an iterative approach to software development that delivers working software in small increments, adapting to changing requirements rather than following a rigid upfront plan. Scrum is a specific Agile framework organizing work into fixed-length iterations called **sprints** (typically 1–2 weeks), with defined roles (Scrum Master, Product Owner, Development Team) and ceremonies (daily stand-up, sprint planning, sprint review, retrospective).

16.2 What is a sprint, and what is a product backlog?

A **sprint** is a fixed time-box during which a set of work is completed and made ready for review. The **product backlog** is a prioritized list of all features, fixes, and work items for the product; items are pulled from it into a **sprint backlog** for a given sprint.

16.3 What is CI/CD, in one line?

Continuous Integration is the practice of frequently merging code changes into a shared branch with automated builds/tests catching problems early. Continuous Delivery/Deployment extends this by automatically preparing (or directly releasing) that tested code to production-like environments, reducing manual release effort.

16.4 What is cloud computing, at a high level?

Renting computing resources (servers, storage, databases) on-demand from a provider (like AWS) over the internet instead of owning physical hardware — you pay for what you use, and can scale resources up or down as needed.

16.5 What is Docker, in one line?

A tool that packages an application together with everything it needs to run (code, runtime, libraries, settings) into a lightweight, portable "container" that runs consistently across any environment.

17. Project-Specific Questions (Prepare in Your Own Words)

Panels weight these heavily — being fluent about your own project usually matters more than reciting definitions. Practice speaking these answers out loud, not just reading them.

17.1 Walk me through the architecture of your project.

Prepare a 60-second answer covering: the overall purpose of the app, the layered structure (Controller → Service → Repository → Database), which frontend technology talks to it and how, and any additional pieces (security, external APIs, batch jobs). Draw it as a simple box diagram if allowed — visual explanations land well in vivas.

17.2 What was your specific role/contribution in the project?

Be specific and honest — name the modules/features you personally built (e.g. "I implemented the user authentication module using Spring Security and JWT" rather than a vague "I worked on the backend"). Panels often ask a deep follow-up on whatever you claim, so only mention what you can defend in detail.

17.3 What challenges did you face, and how did you solve them?

Pick one real, technical challenge (e.g. handling the N+1 query problem, resolving a CORS issue, structuring exception handling) and walk through: what went wrong, how you diagnosed it, and the specific fix. This is one of the highest-value questions to prepare well — it shows problem-solving ability, not memorization.

17.4 Why did you choose Spring Boot for this project?

A good answer references concrete benefits: faster setup via auto-configuration and starters, no need to manually configure a server, built-in production-readiness (Actuator), strong ecosystem for security/data access/testing, and easy integration with a REST-consuming frontend like React.

17.5 How did you handle validation and error handling in your project?

Reference your actual implementation: Bean Validation annotations (`@NotNull` , `@Size` , etc.) on request DTOs with `@Valid` , and a centralized `@RestControllerAdvice` with `@ExceptionHandler` methods returning consistent structured error responses (status code, message, timestamp) instead of default stack traces.

17.6 What database did you use, and why?

State your actual DB (e.g. MySQL) and a reason tied to the project — relational integrity needs (foreign keys, transactions), team familiarity, or a specific feature you needed (full-text search, JSON columns, etc.). If you also touched NoSQL (MongoDB), be ready to explain why that data didn't fit a relational model.

17.7 Did you write tests? Which framework, and what did you test?

Mention JUnit 5 for unit tests and Mockito for mocking dependencies; if you wrote integration tests, mention `@SpringBootTest` / `@DataJpaTest` . Be ready to describe one specific test case you wrote and what scenario it verified.

17.8 How would you deploy this project?

A concise answer is enough given lower priority on DevOps: package as an executable JAR via Maven, containerize with a Dockerfile if used, and deploy to a server or cloud VM/container service — mention CI/CD only if your project genuinely used it, since panels probe further on tools you claim.

17.9 What would you improve if you had more time?

Have 1–2 genuine, specific answers ready (e.g. "add caching for frequently read endpoints," "add more granular role-based access control," "increase test coverage on the service layer"). This shows self-awareness and technical maturity — avoid vague answers like "make it better."

Prepared as a self-study viva companion — answers are written to be spoken aloud in 20–40 seconds each.

Good luck for your viva on **September 18, 2026**. Revisit the ★ high-priority questions the most — they cover roughly 80% of what panels actually ask.